

UTILITY BILLING SYSTEM

Assembly Language Group Project — Project Documentation

Group	Group 3
Course	CSM 254

Group Members

Name	Index Number
Jesse Kwadwo Gyamfi	6160524
Ramsey Selasi Agbenorku	6130724
Samuel Boadi Ampomah	6139124
Eric Kofi Essel	9017923
Enocks Roland Leo	6157124
Palmaak Ngmenmaaloo Campion	6152424
Randy Kwarteng	6166124
Tei Odartey-Fio	6172524
Nana Yaw Takyi Opoku	6175524
Adi Ransford	6127624
Simon Yaw Prempeh Cole	6152824
Nyarkoh Perry	6171824
Sencherey Kwasi Boakye	6182924
Kwasi Gyamfi Fordjour	6158324
Nana Baffoe Etrew-Arhin	6157924
Larbi Nana Kwame Kenneth	6166724
Duah Kwaku Agyemang Jarvis	6155824
Julien Glory Manana	6167724
Festus Kumado	6165124

1. Introduction

For this project, our group built a Utility Billing System entirely in x86 assembly language, running on Linux.

2. Objectives

- Take in a customer's name along with a previous and current meter reading.
- Work out electricity usage and turn that into a bill using a tiered rate, similar to how real utility companies charge.
- Print out a report of every customer entered during that run of the program.

3. Tools and Environment

- Assembler: NASM (Netwide Assembler)
- Linker: GNU ld
- OS: Linux (we targeted both x86 and x86-64)
- We ended up with two builds — a 64-bit one using the modern syscall instruction, and a 32-bit one using the older int 0x80 interrupt. More on why below.

Build commands (32-bit version):

```
nasm -f elf32 billing32.asm -o billing32.o
ld -m elf_i386 billing32.o -o billing32
./billing32
```

We originally wrote the program for 64-bit Linux, using the modern syscall instruction. When we tried assembling it on the online compiler we were given for testing, it failed with address-size errors — turns out that compiler assembles everything as 32-bit by default, and 64-bit registers like rax simply aren't valid there. Rather than switch tools, we ported the whole program over to 32-bit registers and the classic int 0x80 syscall convention that 32-bit Linux programs have always used. Nothing about the logic changed, only how it talks to the kernel.

4. System Design

4.1 Overview

At its core, the program is just a loop with a menu attached to it. Every time round, it prints three options — add a customer reading, view all bills, or exit — reads whatever the operator typed, and jumps to the matching section of code. The tricky part isn't the menu itself, it's everything underneath it: reading a line of text, turning that text into an actual number, turning numbers back into text to display them, and keeping track of a growing list of customers. None of that comes for free in assembly, so we had to write all of it ourselves.

4.2 Customer Record Layout

Each customer is stored as a fixed 32-byte block inside a chunk of memory we reserve up front (basically our version of an array, since assembly doesn't have anything dynamic like a list or a vector). We capped it at 20 customers per run. Here's how one record is laid out:

Field	Offset (bytes)	Size / Purpose
name	0	20 bytes — customer name, null-terminated ASCII string
prev	20	4 bytes (dword) — previous meter reading
curr	24	4 bytes (dword) — current meter reading
bill	28	4 bytes (dword) — computed bill amount

To get to record number i , we just calculate: $\text{record_address} = \text{customers_base} + (i \times 32)$. That's the same thing array indexing does in a higher-level language, just spelled out.

4.3 Program Flow

- `_start`: set `customer_count` to 0 and drop into the main loop.
- `main_loop`: show the menu, read what the operator picked, jump to Add / View / Exit.
- `do_add`: make sure there's room left, ask for name / previous reading / current reading, check the readings make sense, work out the bill, save the record, bump `customer_count` up by one.
- `do_view`: step through every stored record and print the name, usage, and bill.
- `do_exit`: end the program with the exit system call.

5. Core Routines

Assembly doesn't give you string handling or number conversion out of the box, so we wrote our own small set of reusable routines to cover it:

Routine	Registers used	Responsibility
<code>read_line</code>	<code>eax</code> , <code>ebx</code> , <code>ecx</code> , <code>edx</code> , <code>edi</code>	Reads one line of keyboard input, one byte at a time, stopping at the newline character. Saves and restores every register it uses (except <code>esi</code> , deliberately, so callers can keep a pointer alive across the call) so callers keep their state across the call.

str_to_int	eax, ecx, esi	Converts the digit characters produced by read_line into a genuine integer value in eax, by repeatedly multiplying the running total by 10 and adding the next digit.
print_int	eax, ebx, ecx, edx, esi, edi	Converts an integer back into displayable text by repeated division by 10, building the digits in reverse, then writes the result to the screen.
calculate_bill	eax, ebx, ecx, edx	Applies the tiered rate structure to a usage value and returns the total amount due.

6. Billing Algorithm

Usage is just current reading minus previous reading. From there, the bill is worked out in three tiers, the same way a lot of real utility providers do it, so that using more costs progressively more per unit rather than one flat rate:

Tier	Units covered	Rate per unit
Tier 1	First 50 units	5 per unit
Tier 2	Next 100 units (units 51-150)	8 per unit
Tier 3	Units 151 and above	12 per unit

As an example, a customer using 180 units in a period gets billed like this:

- Tier 1: 50 units x 5 = 250
- Tier 2: 100 units x 8 = 800
- Tier 3: the remaining 30 units x 12 = 360
- Total = 250 + 800 + 360 = 1410

calculate_bill handles this by keeping a running total of usage still left to bill, and working through it tier by tier. That way it doesn't matter whether the customer's usage stays inside tier 1, spills into tier 2, or goes all the way into tier 3 — the same routine handles every case.

7. Sample Execution

Below is an actual run of the program (32-bit build), where we add two customers and then view the report — this transcript was re-verified directly against the submitted source code:

```
===== UTILITY BILLING SYSTEM =====
1. Add customer reading
2. View all bills
3. Exit
Choose an option: 1
Customer name: Kwame Mensah
Previous meter reading: 0
Current meter reading: 30
Customer added and bill calculated.
Choose an option: 1
Customer name: Ama Boateng
Previous meter reading: 0
Current meter reading: 180
Customer added and bill calculated.
Choose an option: 2

---- Billing Report ----
Name: Kwame Mensah  Usage: 30  Amount due: GHS 150
Name: Ama Boateng   Usage: 180  Amount due: GHS 1410
```

Choose an option: 3

8. Test Cases

Case	Input	Expected result
Normal entry, tier 1 only	prev=0, curr=30	Usage 30, bill 150 (30 x 5)
Normal entry, spans all tiers	prev=0, curr=180	Usage 180, bill 1410
Invalid reading	curr less than prev (e.g. 50 -> 20)	Program rejects the entry with an error message and returns to the menu
Invalid menu choice	operator enters 9	Program prints 'Invalid option, try again.' and re-displays the menu
View with no data	select option 2 before adding anyone	Program prints 'No customers stored yet.'